

check_cm_workable_items_binary_fresh

check_cm_workable_items_binary_fresh.sh

Revision: 2 Last modified: 2026-09-26T13:05:00Z

Overview

Gate CM-WORKABLE-ITEMS-BINARY-FRESH (pre-build invariant 52). Proves the **shipped, git-tracked** workable-items binary matches the Go sources it was built from.

Status update (2026-09-26)

Invariant 17 no longer picks a binary from disk: it builds from source on every run through scripts/pre_build/build_workable_items_from_source.sh (the BOB-188 operator decision, build on demand). The candidate loop described below is history. This gate remains as defence-in-depth, because other callers still use the shipped binary for as long as it exists.

Why this exists

constitution/scripts/workable-items/bin/workable-items is not a stray build output — it is **deliberately shipped**, so a project that inherits the constitution by reference can run the tool with no Go toolchain

installed. docs/QA_DISCOVERY_LEDGER.md and docs/GOVERNANCE_AUDIT_2026-08-08_ROUND2.md both cite that exact path as the canonical invocation.

Being tracked and executable, it **wins the invariant-17 candidate loop** ahead of any freshly-built sibling — on every fresh clone, for every consumer. So when it drifts behind its source, invariant 17 quietly runs an older build and enforces whatever guard set *that* build happened to contain.

That is BOB-188, measured 2026-08-25:

string	source	tracked bin	current bin
refusing to set terminal status	1	0	1
Issues-location item has TERMINAL status	1	0	1
needle workable-items (proves strings sees)	—	34	34

The needle row is what makes the zeros admissible as evidence rather than a comfortable silence: `strings(1)` reads 34 hits from the *same* tracked binary, so it was not blind — those guard strings were genuinely absent (§11.4.201(7)(b)). A check whose message string is missing from the binary cannot fire, which is why ten un-migrated terminal-status rows survived the gate meant to catch them.

Prerequisites

None to run the gate. Remediating a FAIL needs a Go toolchain matching `go.mod` (currently `go 1.21`; built here with `go1.26.2`).

Usage

```
bash scripts/pre_build/check_cm_workable_items_binary_fresh.sh
```

Runs automatically as pre-build invariant 52.

Verdicts

Exit	Meaning
0	Binary matches its sources, or no binary is shipped (§11.4.69 artifact_not_yet_built)
1	Binary is STALE, carries no fingerprint, or the input scan went BLIND

Remediation

```
cd constitution/scripts/workable-items
GOMAXPROCS=2 nice -n 19 go build -o bin/workable-items ./cmd/
workable-items
cp bin/workable-items bin/workable-items-linux          # kept
                identical by convention
# re-record the fingerprint:
{ find . -name '*.go' -not -path './bin/*' -print; ls go.mod
  go.sum 2>/dev/null | sed 's|^|./|'; } \
  | sort | xargs -r sha256sum | sha256sum | cut -d' ' -f1 >
  bin/.source.sha256
```

Internal behaviour

Fingerprint = sha256 over the sorted sha256sum of every *.go (excluding bin/) plus go.mod/go.sum, persisted at bin/.source.sha256.

Content hash, never mtime (§11.4.86): a touch must not clear staleness and a revert must restore freshness. Mutation M5 asserts exactly that.

Zero inputs is never “clean” (§11.4.201(6)): a find that returns nothing and a source tree with no Go in it produce the identical quiet zero, and only one of them is honest — so an empty scan FAILs as BLIND rather than passing.

Edge cases

- **No binary shipped** → exit 0 with an honest skip. Absence is not staleness.
- **Fingerprint present, binary absent** → exit 0; nothing is being shipped.

- **Binary present, fingerprint absent** → FAIL. A shipped binary that cannot be proven fresh is the precondition of BOB-188, not a lesser state of it.

Open question (operator-owned, §11.4.30)

These binaries are **versioned build artifacts**, which §11.4.30 forbids. They are tracked deliberately, to spare consumers a Go toolchain. Whether to keep shipping them or to require a build is a §11.4.122-class decision the operator owns; it is recorded on BOB-188. This gate takes no position on it — it makes the *staleness* impossible while the decision is pending, which is the part that was actively causing harm.

Related

- `scripts/pre_build/check_cm_served_bundle_fresh.sh` — the sibling freshness gate for the served dashboard bundle (BOB-183). Same class of defect (§11.4.108 SOURCE→ARTIFACT), different artifact, and it uses `--if-present` because a *local* build output may honestly be absent — this one does not, because a *committed* artifact that exists is being shipped.
- `tests/pre_build/test_cm_workable_items_binary_fresh.sh` — 7 paired §1.1 mutations (3 golden-TRUE, 4 golden-FALSE).

Last verified

2026-08-25 — 7/7 mutations pass; gate RED against the stale binary, GREEN after rebuild.